- Requests pass through a middleware component on the way "in," and responses pass through again on the way "out." Resource, action, and result filters are also two-way, though authorization and exception filters run only once for a request, and page filters run three times.
- Middleware can short-circuit a request by returning a response instead of passing it on to later middleware. MVC and page filters can also short-circuit the filter pipeline by returning a response.
- Middleware is often used for cross-cutting application concerns, such as logging, performance profiling, and exception handling. Filters also lend themselves to cross-cutting concerns.

Filters and middleware also differ primarily in three ways:

- Middleware can run for all requests; filters run only for requests that reach the `EndpointMiddleware` and execute a controller action or Razor Page handler.
- Filters have access to MVC constructs such as `ModelState` and `IActionResults`. Middleware in general is independent from MVC and Razor Pages and works at a lower level, so it can't use these concepts.
- Filters can be easily applied to a subset of requests, such as all actions on a single controller or a single Razor Page. Middleware generally applies to all requests that reach a given point in the middleware pipeline.

As for the endpoint filter pipeline, I like to think of middleware versus MVC filters as a question of specificity. Middleware is the more general concept, so it has the wider reach. But if you need to access to MVC constructs or